

---> SEQUELIZE

Client/Postman

↓

Express Route

↓

Controller

↓

Sequelize Model

↓

PostgreSQL

Isme hum **Create, Read, Update, Delete**, validation, error handling aur relationships tak jayenge.

1. Project create karo

Terminal:

```
mkdir sequelize-api
```

```
cd sequelize-api
```

```
npm init -y
```

```
npm install express sequelize pg pg-hstore dotenv
```

```
npm install --save-dev nodemon
```

2. Folder structure

```
sequelize-api/  
|  
├── src/  
|   ├── config/  
|   │   └── database.js  
|   ├── models/  
|   │   └── User.js  
|   ├── controllers/  
|   │   └── userController.js  
|   ├── routes/  
|   │   └── userRoutes.js  
|   └── app.js  
├── .env  
├── .gitignore  
└── package.json
```

3. PostgreSQL database banao

PostgreSQL open karo:

psql postgres

Database: CREATE DATABASE sequelize_db;

Check: \l

Exit: \q

4. .env

Project root me:

PORT=5000

DB_NAME=sequelize_db

DB_USER=postgres

DB_PASSWORD=your_password

DB_HOST=localhost

DB_PORT=5432

Apna actual PostgreSQL password DB_PASSWORD me daalna.

5. Database connection

src/config/database.js

```
const { Sequelize } = require("sequelize");
```

```
const sequelize = new Sequelize(  
  process.env.DB_NAME,  
  process.env.DB_USER,  
  process.env.DB_PASSWORD,  
  {  
    host: process.env.DB_HOST,  
    port: process.env.DB_PORT,  
    dialect: "postgres",  
    logging: false,  
  }  
);
```

```
module.exports = sequelize;
```

Yahan:

```
new Sequelize(...)
```

PostgreSQL ke saath connection banata hai.

6. User Model

src/models/User.js

```
const { DataTypes } = require("sequelize");  
const sequelize = require("../config/database");
```

```
const User = sequelize.define("User", {  
  name: {  
    type: DataTypes.STRING,  
    allowNull: false,  
  },
```

```
  email: {  
    type: DataTypes.STRING,  
    allowNull: false,  
    unique: true,  
  },
```

```
  age: {  
    type: DataTypes.INTEGER,  
    allowNull: false,  
  },  
});
```

```
module.exports = User;
```

Ab samjho:

```
sequelize.define("User", {
```

means:

User table ka structure define karo.

Sequelize automatically id, createdAt, updatedAt bhi add karega by default.

Database roughly:

Users

id

name

email

age

createdAt

updatedAt

7. Controller

Ab actual business logic.

[src/controllers/userController.js](#)

```
const User = require("../models/User");
```

CREATE USER

```
const createUser = async (req, res) => {
  try {
    const { name, email, age } = req.body;

    const user = await User.create({
      name,
      email,
      age,
    });

    res.status(201).json({
      success: true,
      message: "User created successfully",
      data: user,
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }
};
```

GET ALL USERS

```
const getUsers = async (req, res) => {
  try {
    const users = await User.findAll();

    res.status(200).json({
      success: true,
      data: users,
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }
};
```

GET SINGLE USER

```
const getUser = async (req, res) => {
  try {
    const { id } = req.params;

    const user = await User.findByPk(id);

    if (!user) {
      return res.status(404).json({
        success: false,
        message: "User not found",
      });
    }

    res.status(200).json({
      success: true,
      data: user,
    });
  }
};
```

```

    });

    } catch (error) {
      res.status(500).json({
        success: false,
        message: error.message,
      });
    }
  };
};

```

8. UPDATE USER

```

const updateUser = async (req, res) => {
  try {
    const { id } = req.params;
    const { name, email, age } = req.body;

    const user = await User.findById(id);

    if (!user) {
      return res.status(404).json({
        success: false,
        message: "User not found",
      });
    }

    await user.update({
      name,
      email,
      age,
    });

    res.status(200).json({
      success: true,
      message: "User updated successfully",
      data: user,
    });
  }
};

```



```
    } catch (error) {
      res.status(500).json({
        success: false,
        message: error.message,
      });
    }
  };
};
```

9. DELETE USER

```
const deleteUser = async (req, res) => {
  try {
    const { id } = req.params;

    const user = await User.findById(id);

    if (!user) {
      return res.status(404).json({
        success: false,
        message: "User not found",
      });
    }

    await user.destroy();

    res.status(200).json({
      success: true,
      message: "User deleted successfully",
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      message: error.message,
    });
  }
};
```

```
}  
};
```

At the bottom:

```
module.exports = {  
  createUser,  
  getUsers,  
  getUser,  
  updateUser,  
  deleteUser,  
};
```

10. Routes

src/routes/userRoutes.js

```
const express = require("express");  
  
const {  
  createUser,  
  getUsers,  
  getUser,  
  updateUser,  
  deleteUser,  
} = require("../controllers/userController");  
  
const router = express.Router();  
  
router.post("/", createUser);  
  
router.get("/", getUsers);  
  
router.get("/:id", getUser);
```

```
router.put("/:id", updateUser);

router.delete("/:id", deleteUser);

module.exports = router;
```

Now our API is:

```
POST /api/users
GET /api/users
GET /api/users/:id
PUT /api/users/:id
DELETE /api/users/:id
```

11. Main App

src/app.js

```
require("dotenv").config();

const express = require("express");
const sequelize = require("../config/database");

const userRoutes = require("../routes/userRoutes");

const app = express();

app.use(express.json());

app.get("/", (req, res) => {
  res.json({
    message: "Sequelize API is running",
```

```

    });
  });

  app.use("/api/users", userRoutes);

  const PORT = process.env.PORT || 5000;

  const startServer = async () => {
    try {
      await sequelize.authenticate();

      console.log("Database connected successfully");

      await sequelize.sync();

      console.log("Database tables synced");

      app.listen(PORT, () => {
        console.log(`Server running on http://localhost:\${PORT}`);
      });
    } catch (error) {
      console.error("Unable to connect database:", error);
    }
  };

  startServer();

```

12. package.json

Change scripts:

```

{
  "scripts": {
    "start": "node src/app.js",
    "dev": "nodemon src/app.js"
  }
}

```

```
}  
}
```

Run:

npm run dev

You should see:

Database connected successfully

Database tables synced

Server running on <http://localhost:5000>

13. Test with Postman

CREATE

POST

<http://localhost:5000/api/users>

Body → raw → JSON:

```
{  
  "name": "Kuwar",  
  "email": "kuwar@gmail.com",  
  "age": 22  
}
```

Response:

```
{  
  "success": true,  
  "message": "User created successfully",  
  "data": {  
    "id": 1,  
    "name": "Kuwar",  
    "email": "kuwar@gmail.com",  
    "age": 22  
  }  
}
```

```
}  
}
```

GET ALL

GET

<http://localhost:5000/api/users>

Internally:

User.findAll()

GET ONE

GET

<http://localhost:5000/api/users/1>

Internally:

User.findByPk(1)

UPDATE

PUT

<http://localhost:5000/api/users/1>

Body:

```
{  
  "name": "Kuwar Ji",  
  "email": "kuwarji@gmail.com",  
  "age": 23  
}
```

Internally:

```
user.update({  
  name,  
  email,  
  age  
});
```

DELETE

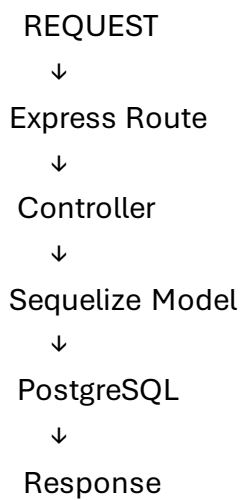
DELETE

<http://localhost:5000/api/users/1>

Internally:

```
user.destroy();
```

Ab Sequelize ka complete flow yaad rakho



Example:

POST /api/users

↓

userRoutes.js

↓

createUser()

↓

User.create()

↓

PostgreSQL

↓

User created

Sequelize ke 5 commands jo sabse pehle yaad karo

User.create()

INSERT

User.findAll()

SELECT ALL

User.findByPk()

SELECT ONE

User.update()

UPDATE

User.destroy()

DELETE